

React

1. ReactJS-HOL

Objectives

- Define SPA and its benefits
- Define React and identify its working
- Identify the differences between SPA and MPA
- Explain Pros & Cons of Single-Page Application
- Explain about React
- Define virtual DOM
- Explain Features of React

In this hands-on lab, you will learn how to:

- Set up a react environment
- Use create-react-app

Prerequisites

The following is required to complete this hands-on lab:

- js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **30 minutes.**

Create a new React Application with the name "myfirstreact", Run the application to print "welcome to the first session of React" as heading of that page.

1. To create a new React app, Install Nodejs and Npm from the following link:

<https://nodejs.org/en/download/>

1. Install Create-react-app by running the following command in the command prompt:
2. To create a React Application with the name of "myfirstreact", type the following command:
3. Once the App is created, navigate into the folder of myfirstreact by typing the following command:
4. Open the folder of myfirstreact in Visual Studio Code
5. Open the App.js file in Src Folder of myfirstreact
6. Remove the current content of "App.js"
7. Replace it with the following:
8. Run the following command to execute the React application:
9. Open a new browser window and type "localhost:3000" in the address bar

Hands-On Lab Instructions

Prerequisites

- Install **Node.js** and **NPM**: <https://nodejs.org/en/download/>
- Install **Visual Studio Code**.

Steps

1. Install **create-react-app** globally (optional step if you don't have it):

```
npm install -g create-react-app
```

2. Create a new React application named **myfirstreact** :

```
npx create-react-app myfirstreact
```

3. Navigate into the project folder:

```
cd myfirstreact
```

4. Open the project in Visual Studio Code:

5. Open `src/App.js` and replace its content with:

```
function App() {  
  return (  
    <div>  
      <h1>Welcome to the first session of React</h1>  
    </div>  
  );  
}  
  
export default App;
```

6. Start the development server:

```
npm start
```

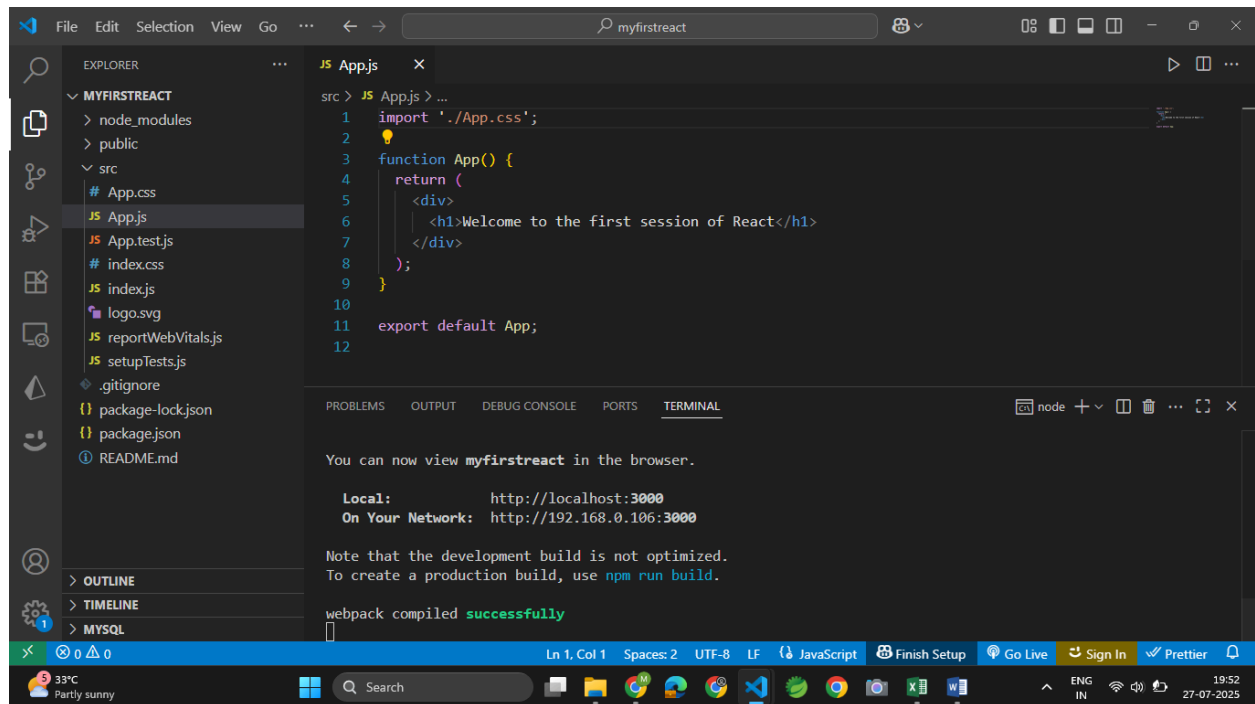
7. Open your browser and visit:

```
http://localhost:3000
```

Output:



Welcome to the first session of React



2. ReactJS-HOL

Objectives

- Explain React components
- Identify the differences between components and JavaScript functions
- Identify the types of components
- Explain class component
- Explain function component
- Define component constructor
- Define render() function

In this hands-on lab, you will learn how to:

- Create a class component
- Create multiple components
- Render a component

Prerequisites

The following is required to complete this hands-on lab:

- js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **30 minutes.**

Create a react app for Student Management Portal named StudentApp and create a component named Home which will display the Message "Welcome to the Home page of Student Management Portal". Create another component named About and display the Message "Welcome to the About page of the Student Management Portal". Create a third component named Contact and display the Message "Welcome to the Contact page of the Student Management Portal". Call all the three components.

1. Create a React project named "StudentApp" type the following command in terminal of Visual studio:

2. Create a new folder under Src folder with the name "Components". Add a new file named "Home.js"
3. Type the following code in Home.js
4. Under Src folder add another file named "About.js"
5. Repeat the same steps for Creating "About" and "Contact" component by adding a new file as "About.js", "Contact.js" under "Src" folder and edit the code as mentioned for "Home" Component.
6. Edit the App.js to invoke the Home, About and Contact component as follows:
7. In command Prompt, navigate into StudentApp and execute the code by typing the following command:
8. Open browser and type "localhost:3000" in the address bar:

1. Create a React project named **StudentApp**

In your Visual Studio Code terminal, run:

```
npx create-react-app StudentApp
```

Then navigate into the project:

```
cd StudentApp
```

2. Create a **Components** Folder

Inside the **src** folder, create a new folder named **Components**.

3. Create **Home.js**

In **src/Components/Home.js**, add:

```
import React, { Component } from 'react';

class Home extends Component {
  render() {
    return (
```

```

    <div>
      <h2>Welcome to the Home page of Student Management Portal</h2>
    </div>
  );
}
}

export default Home;

```

4. Create **About.js**

In **src/Components/About.js**, add:

```

import React, { Component } from 'react';

class About extends Component {
  render() {
    return (
      <div>
        <h2>Welcome to the About page of the Student Management Portal</h2>
      >
      </div>
    );
  }
}

export default About;

```

5. Create **Contact.js**

In **src/Components/Contact.js**, add:

```

import React, { Component } from 'react';

class Contact extends Component {
  render() {

```

```

    return (
      <div>
        <h2>Welcome to the Contact page of the Student Management Portal</h
2>
      </div>
    );
  }
}

export default Contact;

```

6. Edit **App.js** to Call All Components

Open **src/App.js** and replace the content with:

```

import React, { Component } from 'react';
import Home from './Components/Home';
import About from './Components/About';
import Contact from './Components/Contact';

class App extends Component {
  render() {
    return (
      <div className="App">
        <h1>Student Management Portal</h1>
        <Home />
        <About />
        <Contact />
      </div>
    );
  }
}

export default App;

```


7. Run the Application

In the terminal:

```
npm start
```

8. Open in Browser

Go to:

```
http://localhost:3000
```



Student Management Portal

Welcome to the Home page of Student Management Portal

Welcome to the About page of the Student Management Portal

Welcome to the Contact page of the Student Management Portal

3. ReactJS-HOL

Objectives

- Explain React components
- Identify the differences between components and JavaScript functions
- Identify the types of components
- Explain class component
- Explain function component

- Define component constructor
- Define render() function

In this hands-on lab, you will learn how to:

- Create a function component
- Apply style to components
- Render a component

Prerequisites

The following is required to complete this hands-on lab:

- js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **30 minutes**.

Create a react app for Student Management Portal named scorecalculatorapp and create a function component named "CalculateScore" which will accept Name, School, Total and goal in order to calculate the average score of a student and display the same.

1. Create a React project named "scorecalculatorapp" type the following command in terminal of Visual studio:
2. Create a new folder under Src folder with the name "Components". Add a new file named "CalculateScore.js"
3. Type the following code in CalculateScore.js
4. Create a Folder named Stylesheets and add a file named "mystyle.css" in order to add some styles to the components:
5. Edit the App.js to invoke the CalculateScore functional component as follows:
6. In command Prompt, navigate into scorecalculatorapp and execute the code by typing the following command:

7. Open browser and type "localhost:3000" in the address bar:

1. Create the React Project

Open the terminal and run:

```
npx create-react-app scorecalculatorapp  
cd scorecalculatorapp
```

2. Create the **Components** Folder

Inside `src/`, create a folder named **Components**, and then create a file named `CalculateScore.js`.

3. Code for `CalculateScore.js`

```
import React from 'react';  
import '../Stylesheets/mystyle.css'; // Import the CSS file  
  
const percentToDecimal = (decimal) => {  
  return (decimal.toFixed(2) + '%');  
};  
  
const calcScore = (total, goal) => {  
  return percentToDecimal(total / goal);  
};  
  
export const CalculateScore = ({ Name, School, total, goal }) => (  
  <div className="formatstyle">  
    <h1 className="Brown">Student Details:</h1>  
    <div className="Name">  
      <b><span>Name:</span></b>  
      <span>{Name}</span>  
    </div>  
    <div className="School">  
      <b><span>School:</span></b>  
    </div>  
  </div>  
)
```

```

        <span>{School}</span>
    </div>
    <div className="Total">
        <b><span>Total:</span></b>
        <span>{total}</span>
        <span>Marks</span>
    </div>
    <div className="Score">
        <b><span>Score:</span></b>
        <span>
            {calcScore(
                total,
                goal
            )}
        </span>
    </div>
</div>
);

```

4. Create **Stylesheets** Folder

Inside `src/`, create a folder named **Stylesheets**. Add a file named `mystyle.css` with the following content:

```

.Name {
    font-weight: 300;
    color: blue;
}

.School {
    color: crimson;
}

.Total {
    color: darkmagenta;
}

```

```

}

.formatstyle {
  text-align: center;
  font-size: large;
}

.Score {
  color: forestgreen;
}

.Brown {
  color: brown;
}

```

5. Edit **App.js**

Open `src/App.js` and replace the content with:

```

import React from 'react';
import { CalculateScore } from './components/CalculateScore';

function App() {
  return (
    <div>
      <CalculateScore
        Name={"Steeve"}
        School={"DNV Public School"}
        total={284}
        goal={3}
      />
    </div>
  );
}

```

```
export default App;
```

6. Run the Application

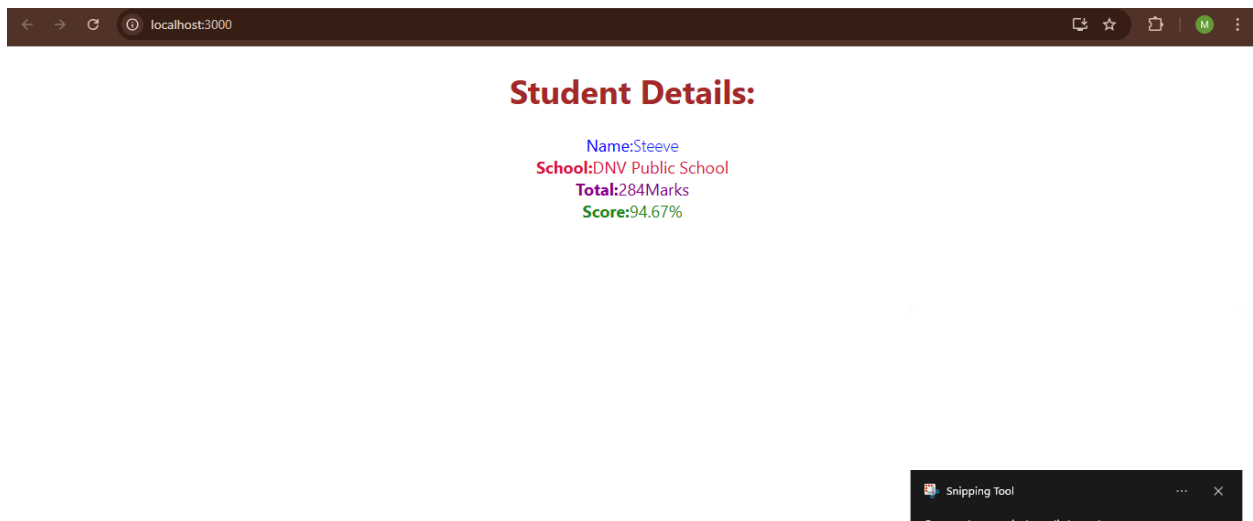
In the terminal:

```
npm start
```

7. Open Browser

Go to:

```
http://localhost:3000
```



4. ReactJS-HOL

Objectives

- Explain the need and Benefits of component life cycle
- Identify various life cycle hook methods

- List the sequence of steps in rendering a component

In this hands-on lab, you will learn how to:

- Implement `componentDidMount()` hook
- Implementing `componentDidCatch()` life cycle hook.

Prerequisites

The following is required to complete this hands-on lab:

- js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **60 minutes**.

1. Create a new react application using *create-react-app* tool with the name as "blogapp"
2. Open the application using VS Code
3. Create a new file named as **js** in **src folder** with following properties

Figure 2: Post class

1. Create a new class based component named as **Posts** inside **js** file

Figure 3: Posts Component

1. Initialize the component with a list of Post in state of the component using the constructor
2. Create a new method in component with the name as **loadPosts()** which will be responsible for using Fetch API and assign it to the component state created earlier. To get the posts use the url (<https://jsonplaceholder.typicode.com/posts>)

Figure 4: loadPosts() method

1. Implement the **componentDidMount()** hook to make calls to **loadPosts()** which will fetch the posts

Figure 5: componentDidMount() hook

1. Implement the **render()** which will display the title and post of posts in html page using heading and paragraphs respectively.

Figure 6: render() method

1. Define a **componentDidCatch()** method which will be responsible for displaying any error happening in the component as alert messages.

Figure 7: componentDidCatch() hook

1. Add the Posts component to App component.
2. Build and Run the application using *npm start*

1. Create a new React application named "blogapp"

Open your terminal or command prompt (e.g., in Visual Studio Code, go to Terminal \> New Terminal) and run the following command:

```
npx create-react-app@latest blogapp
```

This will create a new directory `blogapp` with a basic React project structure.

2. Open the application using VS Code

Once the `create-react-app` command finishes, navigate into your new project and open it with VS Code:

```
cd blogapp
```

3. Create a new file named `Post.js` in the `src` folder with following properties

Create a new file `src/Post.js` and add the `Post` class :

`src/Post.js`

```
class Post {  
  constructor(id, title, body) {
```



```

    this.id = id;
    this.title = title;
    this.body = body;
  }
}

export default Post;

```

4. Create a new class based component named as `Posts` inside `Posts.js` file

Create a new file `src/Posts.js`. This will be your main component for displaying posts.

5. Initialize the component with a list of Post in state of the component using the constructor

`src/Posts.js`

```

import React from 'react';
import Post from './Post';

class Posts extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      posts: [],
      error: null
    };
  }
}

export default Posts;

```

6. Create a new method in component with the name as `loadPosts()` which will be responsible for using Fetch API and assign it to the component state created earlier. To get the posts use the url (<https://jsonplaceholder.typicode.com/posts>)

Add the `loadPosts()` method inside the `Posts` class:

src/Posts.js (continued)

```
import React from 'react';
import Post from './Post';

class Posts extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      posts: [],
      error: null
    };
  }

  loadPosts = async () => {
    try {
      const response = await fetch('<https://jsonplaceholder.typicode.com/p
osts>');
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
      const data = await response.json();
      const fetchedPosts = data.map(item => new Post(item.id, item.title, ite
m.body));
      this.setState({ posts: fetchedPosts });
    } catch (error) {
      console.error("Failed to fetch posts:", error);
      this.setState({ error: error });
    }
  }

  export default Posts;
```

7. Implement the `componentDidMount()` hook to make calls to `loadPosts()` which will fetch the posts

Add the `componentDidMount()` method inside the `Posts` class:

`src/Posts.js` **(continued)**

```
import React from 'react';
import Post from './Post';

class Posts extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      posts: [],
      error: null
    };
  }

  loadPosts = async () => {
    try {
      const response = await fetch('<https://jsonplaceholder.typicode.com/p
osts>');
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
      const data = await response.json();
      const fetchedPosts = data.map(item => new Post(item.id, item.title, ite
m.body));
      this.setState({ posts: fetchedPosts });
    } catch (error) {
      console.error("Failed to fetch posts:", error);
      this.setState({ error: error });
    }
  }

  componentDidMount() {
    this.loadPosts();
  }
}
```

```
}
```

```
export default Posts;
```

8. Implement the `render()` which will display the title and post of posts in html page using heading and paragraphs respectively.

Add the `render()` method inside the `Posts` class:

`src/Posts.js` (continued)

```
import React from 'react';
```

```
import Post from './Post';
```

```
class Posts extends React.Component {
```

```
  constructor(props) {
```

```
    super(props);
```

```
    this.state = {
```

```
      posts: [],
```

```
      error: null
```

```
    };
```

```
  }
```

```
  loadPosts = async () => {
```

```
    try {
```

```
      const response = await fetch('<https://jsonplaceholder.typicode.com/posts>');
```

```
      if (!response.ok) {
```

```
        throw new Error(`HTTP error! status: ${response.status}`);
```

```
      }
```

```
      const data = await response.json();
```

```
      const fetchedPosts = data.map(item => new Post(item.id, item.title, item.body));
```

```
      this.setState({ posts: fetchedPosts });
```

```
    } catch (error) {
```

```
      console.error("Failed to fetch posts:", error);
```

```
      this.setState({ error: error });
```

```

    }
  }

  componentDidMount() {
    this.loadPosts();
  }

  render() {
    const { posts, error } = this.state;

    if (error) {
      return <div>Error: {error.message}</div>;
    }

    if (posts.length === 0) {
      return <div>Loading posts...</div>;
    }

    return (
      <div>
        <h1>Blog Posts</h1>
        {posts.map(post => (
          <div key={post.id} style={{ marginBottom: '20px', border: '1px solid #ccc', padding: '10px' }}>
            <h2>{post.title}</h2>
            <p>{post.body}</p>
          </div>
        ))}
      </div>
    );
  }
}

export default Posts;

```

9. Define a `componentDidCatch()` method which will be responsible for displaying any error happening in the component as alert messages.

Add the `componentDidCatch()` method inside the `Posts` class:

`src/Posts.js` (Full Code)

```
import React from 'react';
import Post from './Post';

class Posts extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      posts: [],
      error: null,
      hasError: false
    };
  }

  loadPosts = async () => {
    try {
      const response = await fetch('<https://jsonplaceholder.typicode.com/p
osts>');
      if (!response.ok) {
        throw new Error(`Network response was not ok, status: ${response.
status}`);
      }
      const data = await response.json();
      const fetchedPosts = data.map(item => new Post(item.id, item.title, ite
m.body));
      this.setState({ posts: fetchedPosts });
    } catch (error) {
      console.error("Error loading posts:", error);
      this.setState({ error: error, hasError: true });
    }
  }
}
```

```

componentDidMount() {
  this.loadPosts();
}

componentDidCatch(error, info) {
  console.error("Caught an error in Posts component:", error, info);
  this.setState({ hasError: true, error: error });
  alert("An error occurred: " + error.message + ". Please try again later.");
}

render() {
  if (this.state.hasError) {
    return (
      <div>
        <h1>Something went wrong.</h1>
        <p>Error details: {this.state.error ? this.state.error.message : 'Unknown error'}</p>
      </div>
    );
  }

  if (this.state.posts.length === 0) {
    return <div>Loading posts...</div>;
  }

  return (
    <div>
      <h1>Blog Posts</h1>
      {this.state.posts.map(post => (
        <div key={post.id} style={{ marginBottom: '20px', border: '1px solid #eee', padding: '15px' }}>
          <h2>{post.title}</h2>
          <p>{post.body}</p>
        </div>
      ))}
    </div>
  );
}

```

```
        </div>
      );
    }
  }

  export default Posts;
```

10. Add the `Posts` component to `App` component.

Open `src/App.js` and modify it to render the `Posts` component:

`src/App.js`

```
import React from 'react';
import Posts from './Posts';

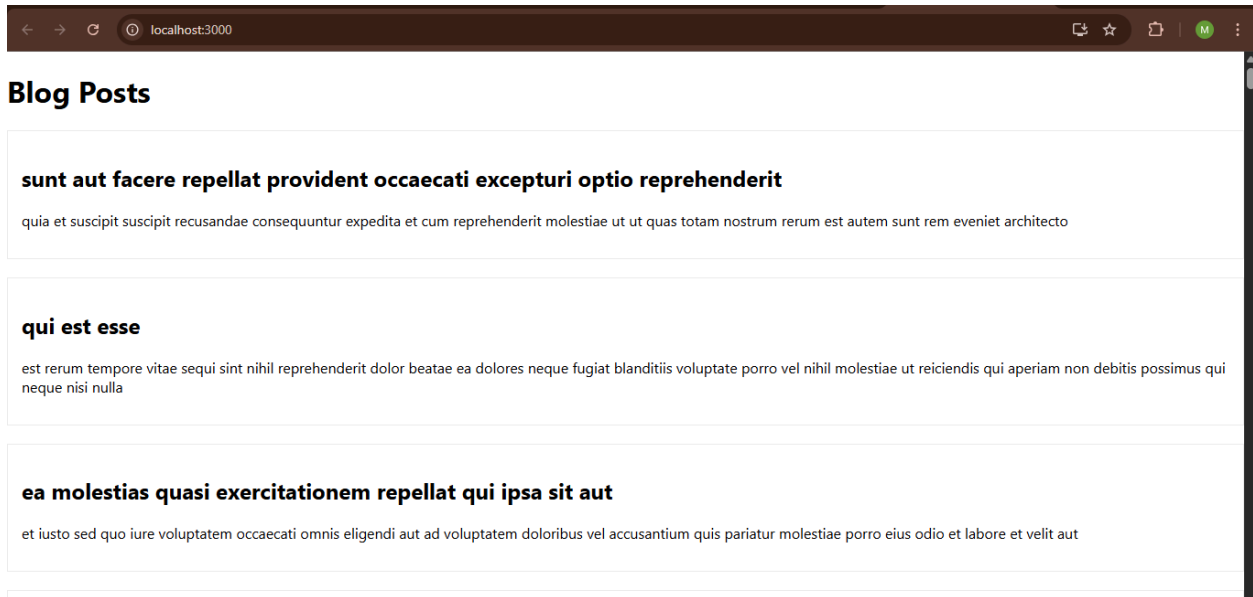
function App() {
  return (
    <div className="App">
      <Posts />
    </div>
  );
}

export default App;
```

11. Build and Run the application using `npm start` command.

Open your terminal in the `blogapp` directory and run:

```
npm start
```

5. ReactJS-HOL

Objectives

- Understanding the need for styling react component
- Working with CSS Module and inline styles

In this hands-on lab, you will learn how to:

- Style a react component
- Define styles using the CSS Module
- Apply styles to components using className and style properties

Prerequisites

The following is required to complete this hands-on lab:

- js
- NPM
- Visual Studio Code

Notes

Estimated time to complete this lab: **30 minutes**.

My Academy team at Cognizant want to create a dashboard containing the details of ongoing and completed cohorts. A react application is created which displays the detail of the cohorts using react component. You are assigned the task of styling these react components.

Download and build the attached react application.

1. Unzip the react application in a folder
2. Open command prompt and switch to the react application folder
3. Restore the node packages using the following commands

Figure 1: Restore packages

1. Open the application using VS Code
2. Create a new CSS Module in a file called "CohortDetails.module.css"
3. Define a css class with the name as "box" with following properties

Width = 300px;

Display = inline block;

Overall 10px margin

Top and bottom padding as 10px

Left and right padding as 20px

1 px border in black color

A border radius of 10px

1. Define a css style for html <dt> element using tag selector. Set the font weight to 500.
2. Open the cohort details component and import the CSS Module
3. Apply the box class to the container div
4. Define the style for <h3> element to use "green" color font when cohort status is "ongoing" and "blue" color in all other scenarios.

5. Final result should look similar to the below image

1. `src/CohortDetails.module.css`

```
.box {  
  width: 300px;  
  display: inline-block;  
  margin: 10px;  
  padding-top: 10px;  
  padding-bottom: 10px;  
  padding-left: 20px;  
  padding-right: 20px;  
  border: 1px solid black;  
  border-radius: 10px;  
  box-shadow: 2px 2px 8px rgba(0, 0, 0, 0.1);  
}  
  
dt {  
  font-weight: 500;  
}
```

2. `src/CohortDetails.js`

```
import React from 'react';  
import styles from './CohortDetails.module.css';  
  
function CohortDetails(props) {  
  const cohortTitleStyle = {  
    color: props.cohort.currentStatus === "Ongoing" ? "green" : "blue"  
  };  
  
  return (  
    <div className={styles.box}>  
      <h3 style={cohortTitleStyle}>
```

```

    {props.cohort.cohortCode} -
    <span>{props.cohort.technology}</span>
  </h3>
  <dl>
    <dt>Started On</dt>
    <dd>{props.cohort.startDate}</dd>
    <dt>Current Status</dt>
    <dd>{props.cohort.currentStatus}</dd>
    <dt>Coach</dt>
    <dd>{props.cohort.coachName}</dd>
    <dt>Trainer</dt>
    <dd>{props.cohort.trainerName}</dd>
  </dl>
</div>
);
}

```

```
export default CohortDetails;
```

←

→

↺

🔍 localhost:3000

Cohorts Details

INTADMDF10 -.NET FSD

Started On

22-Feb-2022

Current Status

Scheduled

Coach

Aathma

Trainer

Jojo Jose

ADM21JF014 -Java FSD

Started On

10-Sep-2021

Current Status

Ongoing

Coach

Apoorv

Trainer

Elisa Smith

CDBJF21025 -Java FSD

Started On

24-Dec-2021

Current Status

Ongoing

Coach

Aathma

Trainer

John Doe

INTADMJF12 -Java FSD

Started On

22-Feb-2022

Current Status

Scheduled

Coach

Ibrahim

Trainer

To Be Assigned

CDE22JF011 -Java FSD

Started On

24-Dec-2021

Current Status

Ongoing

Coach

Apoorv

Trainer

Emma Swan

INTADMDF09 -Dataware Housing

Started On

22-Feb-2022

Current Status

Scheduled

Coach

Aathma

Trainer

Babjee Rao